

PRACTICAL NO 7

Aim: Write a program to implement Tree Traversal of Binary Search Tree.

INPUT:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.leftChild = None
        self.rightChild = None

# preorder (Root → Left → Right)
def preorder(self):
    print(self.data, end=' ')
    if self.leftChild:
        self.leftChild.preorder()
    if self.rightChild:
        self.rightChild.preorder()

# inorder (Left → Root → Right)
def inorder(self):
    if self.leftChild:
        self.leftChild.inorder()
    print(self.data, end=' ')
    if self.rightChild:
        self.rightChild.inorder()

# postorder (Left → Right → Root)
def postorder(self):
    if self.leftChild:
        self.leftChild.postorder()
    if self.rightChild:
        self.rightChild.postorder()
    print(self.data, end=' ')

def insert(root, data):
    # If tree is empty, create a new node
    if root is None:
        return Node(data)
    # Don't insert duplicates
    if data == root.data:
        return root

    if data < root.data:
        root.leftChild = insert(root.leftChild, data)
    else:
        root.rightChild = insert(root.rightChild, data)

    return root
```

```
# Input numbers
numbers = [10, 89, 77, 54, 98, 11, 12, 15, 90]

# First number becomes root
root = None

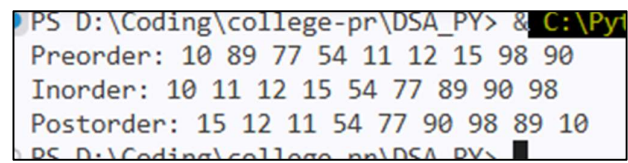
for data in numbers:
    root = insert(root, data)

print("Preorder:", end=' ')
root.preorder()

print("\nInorder:", end=' ')
root.inorder()

print("\nPostorder:", end=' ')
root.postorder()
```

OUTPUT:



```
PS D:\Coding\college-pr\DSA_PY> & C:\Python39\python.exe D:\Coding\college-pr\DSA_PY\main.py
Preorder: 10 89 77 54 11 12 15 98 90
Inorder: 10 11 12 15 54 77 89 90 98
Postorder: 15 12 11 54 77 90 98 89 10
PS D:\Coding\college-pr\DSA_PY>
```